Software Development with AI

Dr. Garrett Dancik

Course material: http://gdancik.github.io

THOUGHTS ABOUT SOFTWARE DEVELOPMENT WITH AI?

State of AI coding

FAST COMPANY

04-23-2026 | WORK LIFE

Google CEO Sundar Pichai says 75% of the company's code is AI-generated

As the tech giant goes all in on AI agents, Google engineers are no longer constrained by time or human energy, Google Cloud's senior director tells Fast Company.

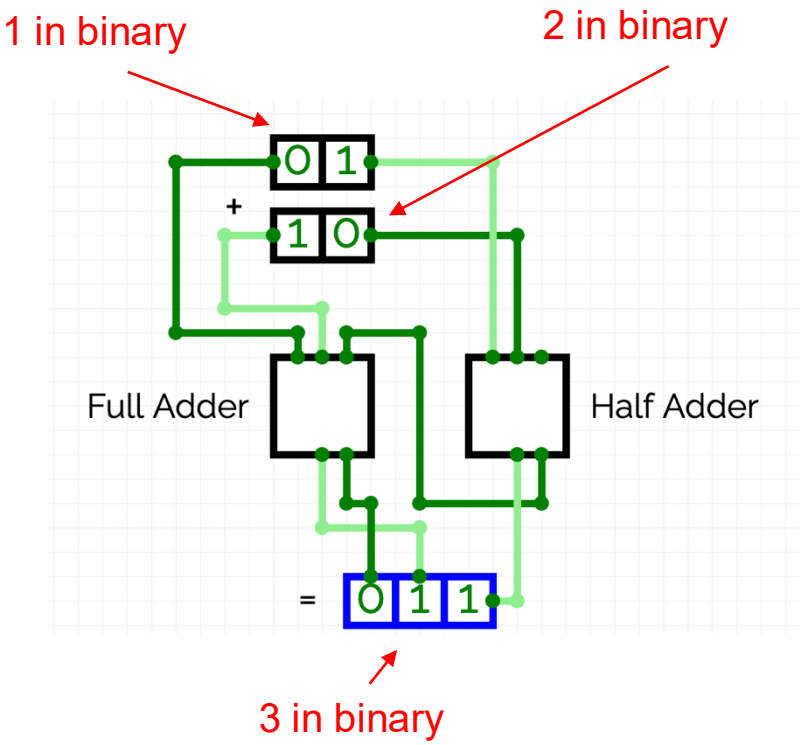
CS is based on abstractions

```java
public static void test() {
  int x = 1 + 2;
}
```

Computer code (Java)

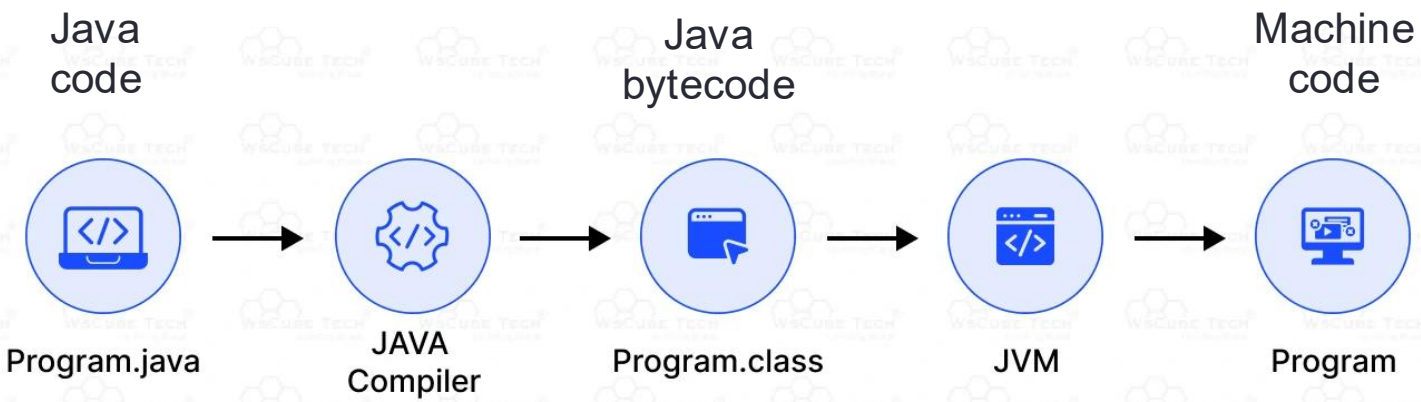
```
; int x = 1 + 2;
mov eax, 1          ; Load 1 into eax

; Add 2 to eax (eax now contains 3)
add eax, 2

; Store result in local variable x (at [rbp-4])
mov [rbp-4], eax
```

Assembly Language



Digital Logic

What happens when you run Java code?



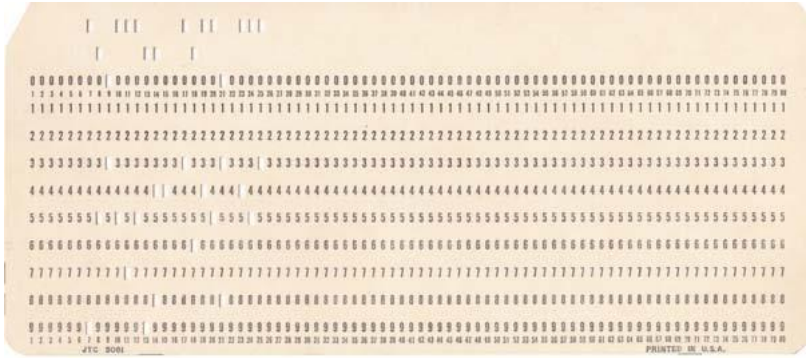
Adapted from: https://www.wscubetech.com/resources/java/compilation-process

WSCUBE TECH® Upskilling Bharat

Machine code

ADD	X	Y
00001001	0000000000001001	0000000000001100
op code for ADD	address of X	address of Y

An evolution of coding



Programs were written using punch cards from the 1950s through the mid 1970s.

Punch cards were used to specify **machine language** (binary code executed directly by the CPU) in the 1940s and then **assembly language** (more human readable commands that map directly to a machine instruction) beginning in the late 1940s.

These **low-level** languages have little/no abstraction from a computer's hardware.

High level languages are designed to be "easy" for humans to read and write. These languages abstract away hardware details required to program in assembly or machine code.

Image source: https://www.columbia.edu/cu/computinghistory/cards.html

An evolution of coding

Select examples of high-level programming languages

Language	Year	Brief description
Fortran	1957	One of the first high-level programming languages, designed for scientific computing.
COBOL	1960	A business-oriented language designed for data processing and business applications.
Prolog	1972	A logic programming language used for artificial intelligence and symbolic reasoning.
C	1972	A general-purpose, compiled language known for speed, efficiency, and low-level memory access.
SQL	1978	A language for working with relational databases.
C++	1980	A general-purpose language originally developed as an extension of C to allow for OOP.
Python	1991	A high-level, general-purpose language emphasizing readability and simplicity.
Java	1995	A general-purpose, object-oriented language designed to "write once, run anywhere."

Computers should adapt to humans

"Very few people are really symbol manipulators… It's much easier for most people to write an English statement than it is to use symbols. So I decided data processors ought to be able to write their programs in English, and the computers would translate them into machine code."

Programmers of the day were extremely resistant to this new technology [compiler-like tools], which could be pre-installed into all computers. "But Grace, then anyone will be able to write programs!" they cried. That was precisely the point..."

Grace Hopper (1906 – 1992)

- creator of the first compiler-like tool
- key influence behind COBOL

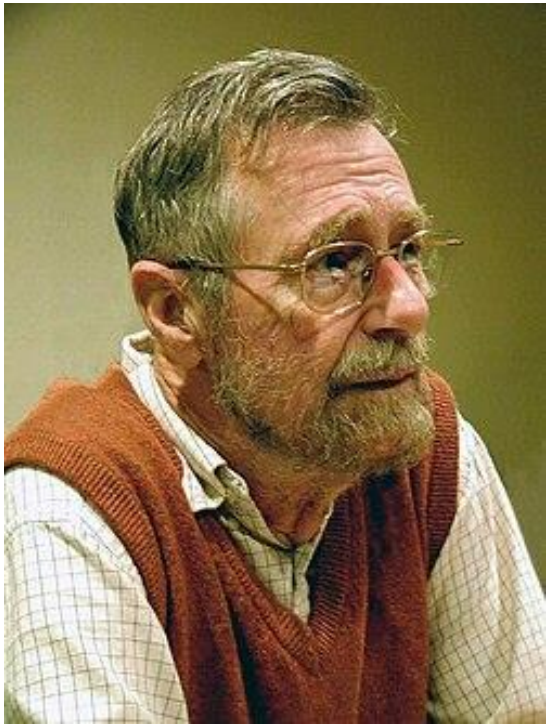
Sources:

- https://www.vassar.edu/stories/2017/assets/images/170706-legacy-of-grace-hopper-hopperpdf.pdf
- https://womenwhocode.com/blog/grace-hopper-pioneering-computer-programmer/

On the foolishness of “natural language programming” (1978)

“In order to make machines significantly easier to use, it has been proposed (to try) to design machines that we could instruct in our native tongues”

“The virtue of formal texts is that…they are…an amazingly effective tool for ruling out all sorts of nonsense that, when we use our native tongues, are almost impossible to avoid.”



Edsger Wybe Dijkstra (1930 – 2002)

- creator of Dijkstra’s algorithm for finding the shortest path

https://www.cs.utexas.edu/~EWD/transcriptions/EWD06xx/EWD667.html

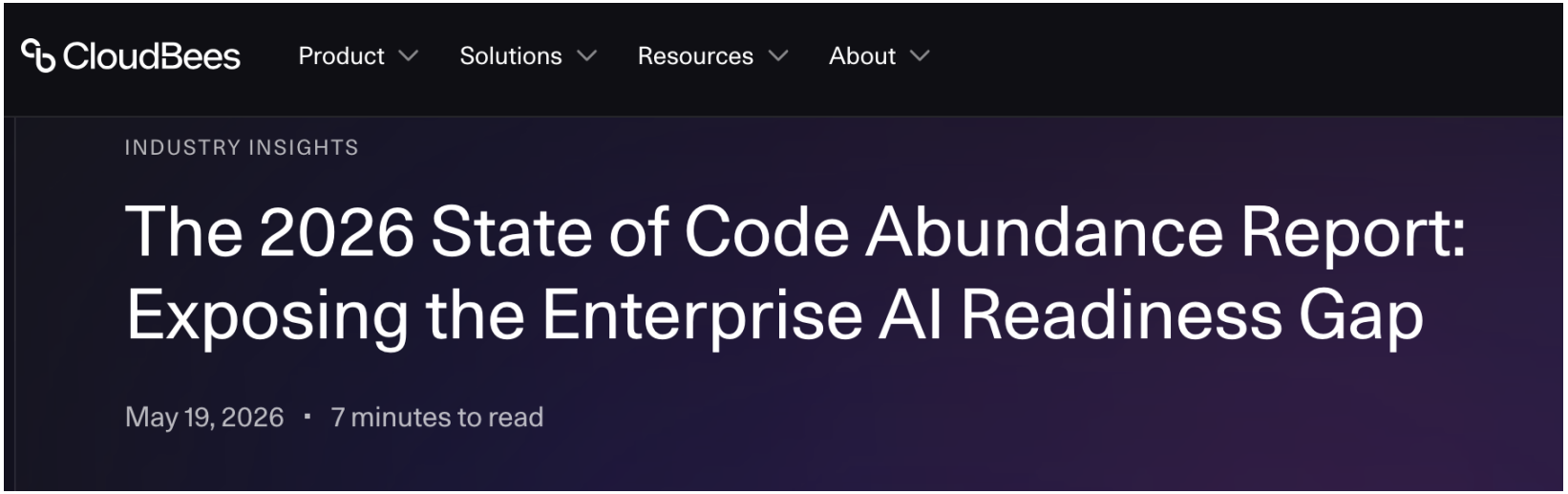
Factorial Example

```
// precondition: a non-negative integer (n) is ready to be specified
// postcondition: returns n! = n*(n-1)*(n-2)*...*1, with 0! = 1.
public static int factorial(int n) {
        int prod = 1;
        for (int i = n; i>1; i--) {
                prod *= i;
        }
        return prod;
}
```

- Can we *prove* that this method is *correct*?
- What if the method is called with a negative number?
- What if the method is called with a very large number?

How do we know that code works?

- Code review: examine code for correctness, logic, readability, design, maintainability, security, etc
- Testing:
  - Does the code compile and run?
  - Are methods and classes logically correct? Do they meet the program specifications?
- Additional concepts covered in this course:
  - Documentation of code
  - Version control using git and github



> "We've never been able to build and write code like this before; it's exhilarating. But we need to remember that code is just an artifact; it's not the actual outcome organizations are pursuing. The real outcome is a quality product. You don't get that until the code is verified, tested, audited, and deployed into the hands of users."
>
> Phil Nash
>
> Developer Relations Engineer - AI, Agents & MCP, Langflow Project

https://www.cloudbees.com/blog/2026-state-of-code-abundance-report

Java Fundamentals

+ 1. Introduction to Java
+ 2. Variables / Assignments
+ 3. Branches
+ 4. Loops
+ 5. Arrays
+ 6. User-Defined Methods
+ 7. Objects and Classes
+ 10. Inheritance
+ 11. Recursion
+ 12. Exceptions
+ 13. Generics
+ 14. Collections

https://www.zybooks.com/catalog/programming-in-java/